

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики-процессов управления

Программа бакалавриата

“Большие данные и распределенная цифровая платформа”

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Алгоритмы и структуры данных»

на тему «Исследование хеш-функций с различными вводными условиями»

Вариант – 10

Студент гр. 23Б15-пу

Трофимов И. А.

Преподаватель

Дик А.Г.

Санкт-Петербург

2024 г.

Оглавление

1. Цель работы	3
2. Теоретическая часть	3
3. Описание задачи	6
4. Основные шаги программы	6
5. Описание программы	7
6. Исходный код программы:	8
7. Блок схема	9
8. Контрольный пример	9
9. Анализ	12
10. Вывод	12
11. Источники	13

Цель работы

Целью лабораторной работы является расшифровать набор данных, зашифрованный с помощью хеш-функции с использованием модификатора входа – соли, а также проанализировать решение аналогичной задачи при различных условиях.

Теоретическая часть

Хеш-функция

Хеш-функция — это алгоритм, который преобразует входные данные произвольной длины в фиксированное значение — хеш, или "отпечаток". Результатом работы хеш-функции является уникальная последовательность символов, которая служит идентификатором исходных данных. Основные характеристики хеш-функций:

1. Односторонность: преобразование данных в хеш легко, но восстановить исходные данные по хешу практически невозможно.
2. Фиксированная длина: независимо от длины исходных данных, хеш-функция всегда возвращает хеш фиксированной длины (например, 160 бит для SHA-1 или 256 бит для SHA-256).
3. Уникальность: малейшее изменение исходных данных приведет к значительному изменению хеша (свойство, называемое "лавинным эффектом").
4. Детерминированность: одно и то же значение всегда приводит к одинаковому хешу.

Хеш-функции широко используются для хранения паролей, проверок целостности данных и в криптографии. Однако они уязвимы к атакам типа Brute Force или с использованием радужных таблиц, если не применяются дополнительные меры, такие как соль.

Техника взлома Brute Force

Brute Force (перебор) — это метод атаки, при котором программа последовательно проверяет все возможные комбинации символов для восстановления исходного значения, соответствующего хешу. Этот метод не требует знаний о структуре данных, но он крайне медленный для сложных паролей или длинных данных.

Основные этапы brute force-атаки:

1. Определение диапазона значений: программа формирует все возможные комбинации символов, которые могут быть использованы в пароле (например, цифры, буквы и специальные символы).
2. Генерация хешей: для каждой возможной комбинации вычисляется хеш.
3. Сравнение хеша: программа сравнивает полученный хеш с целевым хешем. Если хеши совпадают, исходное значение восстановлено.

Для повышения эффективности взлома используют различные стратегии:

- Атака по маске: если известен формат пароля, можно ограничить диапазон поиска (например, если пароль состоит только из цифр).
- Словарные атаки: вместо полного перебора, проверяются значения из заранее составленного списка (словаря) вероятных паролей.

Главный недостаток техники Brute Force — её время выполнения, которое растёт экспоненциально с увеличением длины пароля и числа возможных символов.

Соль

Соль (salt) — это случайное значение, добавляемое к данным перед их хэшированием. Цель соли — усложнить атаки на хеши, такие как Brute Force или атаки с радужными таблицами. Без соли одинаковые значения всегда приводят к одинаковым хешам, что делает их уязвимыми для атак. Соль позволяет этого избежать, даже если несколько пользователей используют одинаковые пароли.

Принцип работы соли:

1. При хэшировании данных (например, пароля) к ним добавляется случайное значение (соль).
2. Полученные данные с солью хэшируются, и результат сохраняется.
3. При проверке пароля соль извлекается и добавляется к введенным данным перед хэшированием для сравнения с хранимым значением.

Преимущества соли:

- Уникальные хеши для одинаковых данных: если два пользователя используют один и тот же пароль, их хеши будут разными благодаря соли.
- Защита от радужных таблиц: радужные таблицы — это предвычисленные хеши для многих значений. Использование соли делает такие таблицы неэффективными, так как каждая комбинация "соль + пароль" требует отдельного расчета.

Описание схемы пошагового выполнения алгоритма

- 1) Загрузка зашифрованного датасета
- 2) Применение алгоритмов для расшифровки данных с использованием метода Brute force

- 3) Тестирование программы с помощью хеш-функции MD5, вследствие чего получаем телефонные номера с солью
- 4) С помощью скрипта на python находим соль для этих номеров
- 5) Далее, благодаря скрипту на python зашифровываем номера через SHA1, bcrypt
- 6) Применение алгоритмов для расшифровки данных с использованием метода Brute force

Описание задачи

Задача состоит в расшифровке датасета с номерами телефонов, с последующим анализом аналогичной задачи с различными входными данными:

- 1) Программа должна считывать входной файл (hashes.txt) с хешами и расшифрованными номерами телефонов без соли и разбить его на два файла cracked.txt и phones.txt.
- 2) Расшифровка датасета с последующим нахождением соли.
- 3) Зашифровать датасет с номерами функциями разных семейств, и протестировать расшифровку с разными солями.
- 4) Найти минимальное количество телефонов для нахождения соли.

Основные шаги программы

1. **Запуск программы:** Пользователь запускает скрипт для считывания входного файла, и загружаются два файла hashes.txt и phones.txt.
2. **Расшифровка хэшей:** С помощью утилиты hashcat[\[1\]](#) расшифровать хэши и сохранить их в файл cracked.txt.

3. **Нахождение соли:** С помощью python скрипта находится соль и сохраняется файл с датасетом без соли.
4. **Тестирование задачи:** Задача тестируется с различными солями и хеш-функциями.

Описание программы

Программная реализация написана на языке Python 3.12.2 с использованием библиотеки hashlib[\[2\]](#) и утилиты hashcat[\[1\]](#). Задача выполняется посредством последовательного выполнения скриптов и команд hashcat[\[1\]](#).

Таблица1 Скрипты

Скрипт	Описание	Возвращает
command.txt	Содержит скрипты hashcat	phones.txt, hashes.txt
find_salt.py	Находит соль для расшифрованного файла с помощью известных номеров телефона и записывает сырые номера телефонов в файл.	Соль, right_phones.txt
main.py	Хеширует телефоны различными функциями с различными солями	hash_<hash>.txt
plan.md	Подробно описание запуска всех команд и обработки результата	None

Исходный код программы:

[Labs/3 at commt-5 · hanglider/Labs](#)

Рекомендации для пользователя

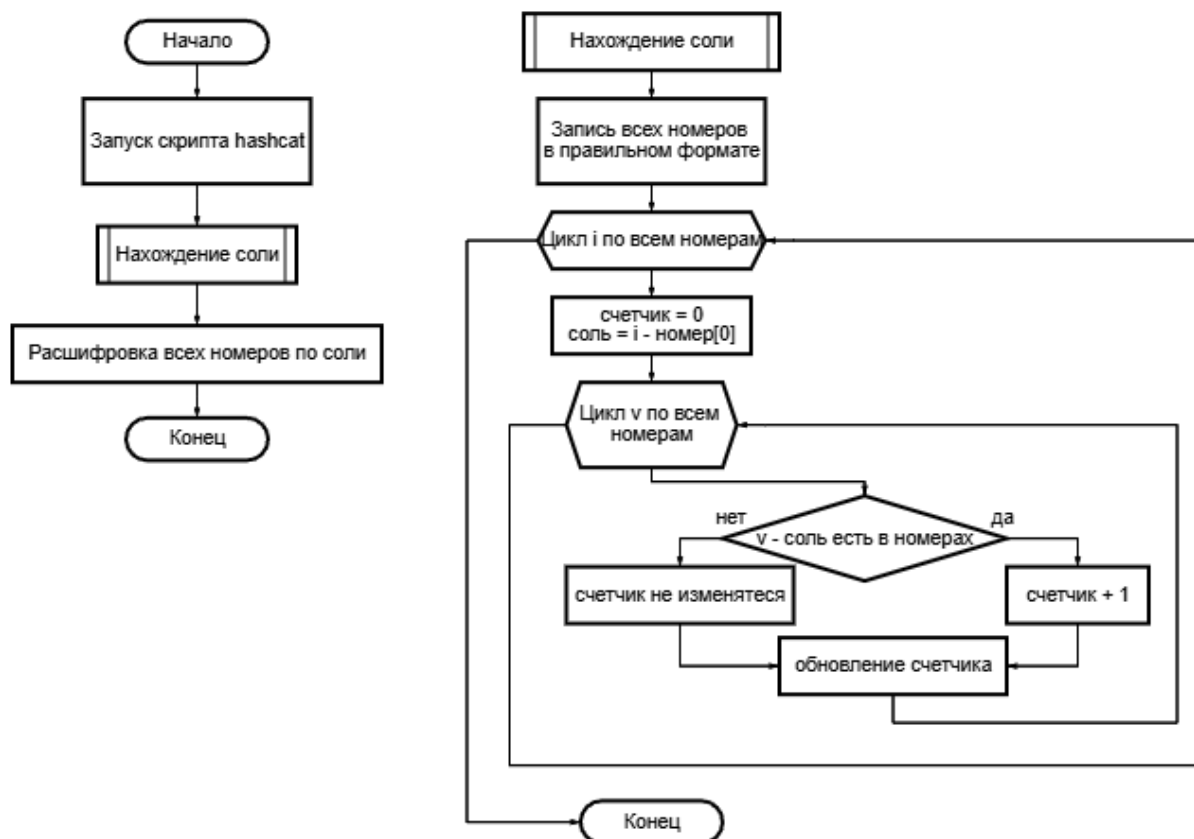
1. Рекомендуется использовать сложные и длинные пароли, которые включают комбинации букв, цифр и символов. Это значительно усложняет перебор значений методом Brute force.
2. При регистрации на сайтах или использовании приложений убедитесь, что они используют современные хеш-функции и механизм соли для защиты данных. Обратите внимание на использование SHA-256, bcrypt , так как они обеспечивают более высокий уровень защиты.
3. Регулярно обновляйте свои пароли и используйте уникальные пароли для разных сервисов. Это снизит риск взлома при утечке данных с одного ресурса.

Рекомендации для программиста

1. Использовать соль при хешировании. Соль должна быть безопасной и уникальной для каждого хешируемого значения. Это защитит от атак с использованием радужных таблиц.
2. Убедитесь, что соль сохраняется и используется корректно. Соль можно хранить вместе с хешем, но она должна быть уникальной для каждого пользователя, чтобы атаки на один хеш не помогли взломать другие данные.
3. Вводите меры защиты от попыток перебора (Brute force) паролей. Это может быть ограничение количества попыток входа, введение капчи или временная блокировка аккаунта после нескольких неудачных попыток.

4. Периодически проверяйте и обновляйте алгоритмы и параметры хеширования, чтобы они соответствовали современным стандартам безопасности.

Блок схема



Контрольный пример

1. Считывание входного файла

Для считывания входного файла запустите скрипт из файла `command.txt` для md5 или следуйте указаниям `plan.md`.

2. Расшифровка

Расшифровка хешей утилитой `hashcat`[\[1\]](#) с помощью атаки по маске (Рис. 1 и 2).

```
C:\IT\Labs\Labs\hashcat-6.2.6>hashcat -m 0 -a 3 -O --opencl-device-types 2 -w 3 C:\IT\Labs\Labs\waste\hashes.txt 89?d?d?d?d?d?d?d?d?d?d -o C:\IT\Labs\Labs\waste\cracked.txt
```

Рисунок 1. Пример команды для расшифровки хэшей MD5

```
Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 100 (SHA1)
Hash.Target.....: C:\IT\Labs\Labs\waste\hash_sha1.txt
Time.Started.....: Thu Oct 24 14:58:03 2024 (5 mins, 0 secs)
Time.Estimated...: Thu Oct 24 15:03:03 2024 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Mask.....: 89?d?d?d?d?d?d?d?d [11]
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 3356.4 kH/s (14.21ms) @ Accel:1024 Loops:100 Thr:32 Vec:1
Recovered.....: 48925/48925 (100.00%) Digests (total), 48925/48925 (100.00%) Digests (new)
Remaining.....: 0 (0.00%) Digests
Recovered/Time...: CUR:9990,N/A,N/A AVG:9773.74,N/A,N/A (Min,Hour,Day)
Progress.....: 1000000000/1000000000 (100.00%)
Rejected.....: 0/1000000000 (0.00%)
Restore.Point....: 9928704/10000000 (99.29%)
Restore.Sub.#1...: Salt:0 Amplifier:0-100 Iteration:0-100
Candidate.Engine.: Device Generator
Candidates.#1....: 89129038737 -> 89396497383
Hardware.Mon.#1..: Temp: 0c Fan: 0% Util: 0% Core: 400MHz Mem: 400MHz Bus:16

Started: Thu Oct 24 14:58:01 2024
Stopped: Thu Oct 24 15:03:04 2024
```

Рисунок 2. Результат расшифровки входного датасета

3. Нахождение соли

Запуск скрипта `find_salt.py`, который находит соль и записывает номера телефонов без соли в отдельный файл `right_phones.txt`. Мною была получена соль - “41224301”

```
PS C:\IT\Labs\Labs> & C:/Users/Admin/AppData/Local/Microsoft/WindowsApps/python3.13.exe c:/IT/Labs/Labs/3/find_salt.py
41224301
```

Рис 3. Соль, найденная скриптом

4. Тестирование задачи с различными солями и хеш-функциями

Запуска скрипта `hashing.py`, который хэширует исходный датасет с различными солями различными хеш-функциями и сохраняет зашифрованные номера телефонов в отдельные файлы. Далее с помощью утилиты `hashcat`[\[1\]](#) происходит расшифровка хешей.

```

Session.....: hashcat
Status.....: Running
Hash.Mode.....: 3200 (bcrypt $2*$, Blowfish (Unix))
Hash.Target.....: C:\IT\Labs\Labs\waste\hash_bcrypt.txt
Time.Started.....: Thu Oct 24 16:16:52 2024 (54 secs)
Time.Estimated...: Next Big Bang (> 10 years)
Kernel.Feature...: Pure Kernel
Guess.Mask.....: 89?d?d?d?d?d?d?d [11]
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 31 H/s (12.73ms) @ Accel:4 Loops:8 Thr:16 Vec:1
Recovered.....: 0/1000 (0.00%) Digests (total), 0/1000 (0.00%) Digests (new), 0/1000 (0.00%) Salts
Progress.....: 1536/100000000000000 (0.00%)
Rejected.....: 0/1536 (0.00%)
Restore.Point...: 0/10000000000 (0.00%)
Restore.Sub.#1...: Salt:8 Amplifier:0-1 Iteration:1976-1984
Candidate.Engine.: Device Generator
Candidates.#1...: 89121999999 -> 89001999999
Hardware.Mon.#1..: Temp: 0c Fan: 0% Util:100% Core:1900MHz Mem: 800MHz Bus:16

```

Рис 4. Результат расшифровки датасета с хеш-функцией bcrypt и солью 1

```

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 100 (SHA1)
Hash.Target.....: C:\IT\Labs\Labs\waste\hash_sha1.txt
Time.Started.....: Thu Oct 24 14:58:03 2024 (5 mins, 0 secs)
Time.Estimated...: Thu Oct 24 15:03:03 2024 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Mask.....: 89?d?d?d?d?d?d?d?d [11]
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 3356.4 kH/s (14.21ms) @ Accel:1024 Loops:100 Thr:32 Vec:1
Recovered.....: 48925/48925 (100.00%) Digests (total), 48925/48925 (100.00%) Digests (new)
Remaining.....: 0 (0.00%) Digests
Recovered/Time...: CUR:9990,N/A,N/A AVG:9773.74,N/A,N/A (Min,Hour,Day)
Progress.....: 1000000000/1000000000 (100.00%)
Rejected.....: 0/1000000000 (0.00%)
Restore.Point...: 9928704/100000000 (99.29%)
Restore.Sub.#1...: Salt:0 Amplifier:0-100 Iteration:0-100
Candidate.Engine.: Device Generator
Candidates.#1...: 89129038737 -> 89396497383
Hardware.Mon.#1..: Temp: 0c Fan: 0% Util: 0% Core: 400MHz Mem: 400MHz Bus:16

Started: Thu Oct 24 14:58:01 2024
Stopped: Thu Oct 24 15:03:04 2024

```

Рис 5. Результат расшифровки датасета с хеш-функцией SHA1 и солью 1357

```

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 100 (SHA1)
Hash.Target.....: C:\IT\Labs\Labs\waste\hash_sha1.txt
Time.Started.....: Thu Oct 24 14:58:03 2024 (5 mins, 0 secs)
Time.Estimated...: Thu Oct 24 15:03:03 2024 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Mask.....: 89?d?d?d?d?d?d?d [11]
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 3356.4 kH/s (14.21ms) @ Accel:1024 Loops:100 Thr:32 Vec:1
Recovered.....: 48925/48925 (100.00%) Digests (total), 48925/48925 (100.00%) Digests (new)
Remaining.....: 0 (0.00%) Digests
Recovered/Time...: CUR:9990,N/A,N/A AVG:9773.74,N/A,N/A (Min,Hour,Day)
Progress.....: 1000000000/1000000000 (100.00%)
Rejected.....: 0/1000000000 (0.00%)
Restore.Point...: 9928704/10000000 (99.29%)
Restore.Sub.#1...: Salt:0 Amplifier:0-100 Iteration:0-100
Candidate.Engine.: Device Generator
Candidates.#1...: 89129038737 -> 89396497383
Hardware.Mon.#1..: Temp: 0c Fan: 0% Util: 0% Core: 400MHz Mem: 400MHz Bus:16

Started: Thu Oct 24 14:58:01 2024
Stopped: Thu Oct 24 15:03:04 2024

```

Рис 6. Результат расшифровки датасета с хеш-функцией sha1 и солью 1

```

Session.....: hashcat
Status.....: Running
Hash.Mode.....: 3200 (bcrypt $2*$, Blowfish (Unix))
Hash.Target.....: C:\IT\Labs\Labs\waste\hash_bcrypt.txt
Time.Started.....: Thu Oct 24 16:16:52 2024 (54 secs)
Time.Estimated...: Next Big Bang (> 10 years)
Kernel.Feature...: Pure Kernel
Guess.Mask.....: 89?d?d?d?d?d?d?d [11]
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 31 H/s (12.73ms) @ Accel:4 Loops:8 Thr:16 Vec:1
Recovered.....: 0/1000 (0.00%) Digests (total), 0/1000 (0.00%) Digests (new), 0/1000 (0.00%) Salts
Progress.....: 1536/1000000000000 (0.00%)
Rejected.....: 0/1536 (0.00%)
Restore.Point...: 0/1000000000 (0.00%)
Restore.Sub.#1...: Salt:8 Amplifier:0-1 Iteration:1976-1984
Candidate.Engine.: Device Generator
Candidates.#1...: 89121999999 -> 89001999999
Hardware.Mon.#1..: Temp: 0c Fan: 0% Util:100% Core:1900MHz Mem: 800MHz Bus:16

```

Рис 7. Результат расшифровки датасета с хеш-функцией bcrypt и солью 12345678

Анализ

Время работы расшифровки хешей утилитой hashcat[\[1\]](#) зависит напрямую от вида функции. В рамках данной задачи, на скорость расшифровки не влияет длина соли, в случае сложения/вычитания соли, но на время выполнения влияет длина соли в случае конкатенации. Минимальное количество телефонов для нахождения соли – 3.

Вывод

В рамках данной работы был разработан алгоритм для расшифровки датасета с номерами телефонов. Реализованный алгоритм обеспечивает возможность анализа выполнения задачи на различных данных, что помогает лучше понять работу хеш-функций и метода brute-force.

Источники

1. hashcat's documentation // hashcat URL: <https://hashcat.net/wiki/> (дата обращения: 20.10.2024).
2. Python hashlib documentation // hashlib URL: <https://docs.python.org/3/library/hashlib.html> (дата обращения: 20.10.2024).